

sources_1\new\STATE_MACHINE.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  --*****
6  --*
7  --* Name: STATE_MANAGER
8  --* Designer: Matthew Mwangi
9  --*
10 --*   This component handles three tasks. First, it manages the synchronization of the
11 --*   reset, player 1 and 2's buttons through the use of the imported INPUT_SYNC and
12 --*   RESET SYNC components. Second it manages the different game states and their
13 --*   corresponding transitions. Third, it takes care of displaying the ball's position
14 --*   through the use of the leds marked as output.
15 --*
16 --*   Once the button inputs are synchronized the STATE_MANAGER process is tasked with
17 --*   performing a number of actions depending on the game's currentState. The possible
18 --*   game states are PLAYER_ONE, MOVE_BALL and PLAYER_TWO, the PLAYER_ONE STATE serves
19 --*   as the beggining game state on reset or start of the game. When p1BtnSync is ACTIVE
20 --*   when in PLAYER_ONE game state a transition is made to MOVE_BALL state where the
21 --*   ball movement is handled. This is done by having a count variable the starts at 15
22 --*   (player 1's side) and counts down to 0 (player 2's side) through the use of a
23 --*   direction variable that as the name suggest dictates the ball's move direction. Once
24 --*   the count variable counts down to 0 the state changes to PLAYER_TWO where player 2
25 --*   gets a chance to shoot the ball through the p2BtnSync input. If the ball is shot then
26 --*   the state changes back to MOVE_BALL where the opposite of what happened earlier
27 --*   (count variable counts up to 15) occurs and Once 15 is reached the state switches to
28 --*   the PLAYER_ONE state. The ball's positions is shown to the players through the leds.
29 --*
30 --*   When either player 1 or player 2 miss to take the shot at their turn then the player
31 --*   that took the shot last gets a point which gets stored in the score signal(5 downto
32 --*   0) where the upper 3 bits is the player 1's score and lower 3 bits is Player 2's
33 --*   score. That player also gets tonext.
34 --*****
35
36 entity STATE_MANAGER is
37 Port(
38     p1Btn:    in std_logic;      --player 1 (left btn)

```

```

39     p2Btn:    in std_logic;        --player 2 (right btn)
40
41     reset:    in std_logic;        --reset the game (center btn)
42     clock:    in std_logic;        --system clock
43     clkTick:  in std_logic;        --A pulse at 10HZ that serves as the LEDs clock
44
45     ballPos:  out std_logic_vector(3 downto 0); -- ball's current position
46     scoreSigP1: out std_logic; --signal sent when player 1 score
47     scoreSigP2: out std_logic --signal sent when player 2 score
48 );
49 end STATE_MANAGER;
50
51
52 architecture STATE_MANAGER_ARCH of STATE_MANAGER is
53
54     constant BLANK_LEDS: std_logic_vector(15 downto 0) := "0000000000000000"; --Sets all LEDs to LOW
55
56     constant ACTIVE: std_logic := '1';
57
58     type States_t is (PLAYER_ONE, MOVE_BALL, PLAYER_TWO); --Possible Game Loop States
59     signal currentState: States_t; --Keeps track of the Current Game Loop State
60     signal scrUpdate: std_logic;
61
62     signal scrOnP1: std_logic_vector(1 downto 0); --goes active when player 1 got scored on
63     signal scrOnP2: std_logic_vector(1 downto 0); --goes active when player 2 got scored on
64
65     signal pos: integer range 0 to 15; --Keeps track of the led position
66
67 begin
68
69 -----
70 ----- BLOCK C -----
71 --* STATE_MANAGER handles the logic behind the three gameloop states
72 --* (PLAYER_ONE, MOVE_BALL, PLAYER_TWO) and updates the pos signal appropriately.
73 -----
74 STATE_MANAGER: process(reset, clock, clkTick)
75     variable direction: integer range -1 to 1; --Serves as the ball direction indicator when
76         --When direction = -1 then ball position goes from player 1 downto player 2
77         --When direction = 0 then stall at currentState
78         --When direction = 1 then ball position goes from player 2 to player 1
79     variable count: integer range 0 to 15 ; --Ball's Position
80

```

```

81 begin
82
83 if reset = ACTIVE then    --Detect any ACTIVE reset or gameOver signals
84     currentState <= PLAYER_ONE;    --Sets the begin State to PLAYER_ONE
85     scrUpdate <= not ACTIVE;
86     scrOnP1 <= (others=>'0');
87     scrOnP2 <= (others=>'0');
88     direction := 0;    --Set direction to zero
89 elsif rising_edge(clock) then
90     scrOnP1 <= scrOnP1(0) & not ACTIVE; --set scrOnP1 to a recurring low on every rising clock edge
91     scrOnP2 <= scrOnP2(0) & not ACTIVE; --set scrOnP2 to a recurring low on every rising clock edge
92     case currentState is
93         -----BUBBLE D: PLAYER_ONE-----
94         when PLAYER_ONE =>
95             count := 15;    -- Set ball pos to player one
96             if(p1Btn = ACTIVE) then    --Check if Player 1 has hit the ball
97                 direction := -1;    --updates the ball's move direction to Player 2
98                 currentState <= MOVE_BALL;    --State change to MOVE_BALL
99             else
100                 if clkTick = ACTIVE and direction = 1 then    --missed shot -. send to Player 2 to start the game
101                     direction := 0;    --Stall Indicator
102                     scrOnP1 <= ACTIVE & scrOnP1(0);    --set scrOnP1 to active
103                     currentState <= PLAYER_TWO;    --Player 1 gives up its turn to serve to player 2
104                 else
105                     currentState <= PLAYER_ONE; --Stalls at PLAYER_ONE when no state transition requirement is met
106                 end if;
107             end if;
108
109         ----- BUBBLE E: MOVE_BALL -----
110         when MOVE_BALL =>
111             if clkTick = ACTIVE then    --Updates ball's position only on the clkTick pulse
112                 if((count = 15) and (direction > 0)) then    --Check whether the ball has reached player 1's side
113                     currentState <= PLAYER_ONE;    --if so change currentState to PLAYER_ONE
114                 elsif ((count = 0) and (direction < 0)) then    --Check whether the ball has reached player 2's side
115                     currentState <= PLAYER_TWO;    --if so change currentState to PLAYER_TWO
116                 end if;
117                 count := count + direction; --update's the position with respect to the direction set
118             else
119                 currentState <= MOVE_BALL;    --Keeps updating ball position otherwise
120             end if;
121
122         -----BUBBLE F: PLAYER_TWO-----

```

```

123     when PLAYER_TWO =>
124         count := 0; --Sets the ball position to player 1
125     -- ballPos <= std_logic_vector(to_unsigned(count, ballPos'length));
126     if(p2Btn = ACTIVE) then --Checks if player 2 hits the ball
127         direction := 1; --Updates the ball's move direction back to Player 1
128         currentState <= MOVE_BALL; --State change to MOVE_BALL
129     else
130         if direction /= 0 then --Checks the ball's incoming direction
131             if clkTick = ACTIVE then --missed shot -. send to Player 1 to start the game
132                 currentState <= PLAYER_ONE;
133                 scrOnP2 <= ACTIVE & scrOnP2(0); --set scrOnP2 to active
134             end if;
135         else
136             currentState <= PLAYER_TWO; --Stall at current state (happens when direction = 0)
137         end if;
138     end if;
139 end case;
140
141 scoreSigP1 <= scrOnP2(1); --activates only for 1 clock cycle when scoreSigP1 when player 1 scores
142 scoreSigP2 <= scrOnP1(1); --activates only for 1 clock cycle when scoreSigP2 when player 2 scores
143 ballPos <= std_logic_vector(to_unsigned(count, ballPos'length)); --store ball's position
144 end if;
145 end process STATE_MANAGER;
146
147 end STATE_MANAGER_ARCH;
148

```